

Datum: 4. Juni 2026 | Projekt: VaadinExcelExport (github.com/MaKnoNet/VaadinExcelExport)

1 Benchmark-Ergebnis

Messung auf demselben Grid (SampleGrid) mit 25.000 Zeilen, 2 Warmup-Läufen und 5 gemessenen Läufen; rein in-memory (ByteArrayOutputStream), kein UI/Browser.

Engine	Median	Avg	Zeilen/s	Groesse
xlsbuilder	237,5 ms	236,9 ms	105.249	850 KB
Flowingcode	5.564,7 ms	5.582,9 ms	4.493	727 KB

xlsbuilder ist ~23x schneller

Konfiguration: Java 21, Gradle 8.9, Vaadin 24.5.3, Apache POI 5.4.0 | Ausfuehrung: ./gradlew benchmark [-Prows=N] | Klasse: ExcelExporterBenchmarkTest

2 Feature-Vergleich

2.1 Ausgabeformate

Feature	xlsbuilder	Flowingcode
Excel (.xlsx)	✓	✓
CSV	✗	✓
DOCX (Word)	✗	✓
PDF	✗	✓

2.2 Datenquelle, Sortierung & Filter

Feature	xlsbuilder	Flowingcode
Respektiert Grid-Sortierung (Spaltenköpfe)	✓	✓
Respektiert aktiven Grid-Filter	✗	✓
TreeGrid / hierarchische Daten	✗	✓
Streaming / out-of-core (konstanter Speicher)	✓	✗

Beide Exporte folgen der aktuellen Tabellensortierung: xlsbuilder übernimmt dazu den In-Memory-Comparator des Grids in die Query, Flowingcode liest ihn aus dem DataCommunicator. Unterschied bleibt beim **Filter**: Flowingcode exportiert nur die gefilterten (sichtbaren) Zeilen, xlsbuilder exportiert den vollständigen Datenbestand.

2.3 Zelltypen & Formatierung

Feature	xlsbuilder	Flowingcode
Typisierte Zellen (INTEGER, LONG, DOUBLE, DECIMAL, BOOLEAN, DATE, DATETIME, TIME)	✓	✗
Echte Excel-Formeln (z.B. =E2*0.19)	✓	✗
Klickbare Hyperlinks in Excel	✓ (HYPERLINK)	✗ (URL als Text)
Excel-Formatcode pro Spalte (#,##0.00)	✓	✓
Java DecimalFormat/DateFormat + Excel-Code kombiniert	✗	✓
Null-Wert-Handler	✗	✓

Zu „Java DecimalFormat/DateFormat + Excel-Code kombiniert“: Flowingcode kann pro Spalte gleichzeitig ein Java-Format-Objekt UND einen Excel-Formatcode setzen, z.B. `setNumberColumnFormat(col, new DecimalFormat("#, ##0.00"), "#, ##0.00")`. Das Java-Format formatiert den Wert als Text für die Nicht-Excel-Ausgaben (CSV, DOCX, PDF), der Excel-Code formatiert die Zelle in der .xlsx – das gibt über alle vier Ausgabeformate konsistente Darstellung. xlsbuilder erzeugt nur .xlsx und nutzt allein den Excel-Formatcode (`formatForType("#, ##0.00")`); ein zusätzliches Java-Format ist dort nicht nötig und daher bewusst nicht vorhanden.

2.4 Template & Layout

Feature	xlsbuilder	Flowingcode
Einfacher Sheet-Name	✓	✓
Template-basierter Export (.xlsx-Vorlage mit Platzhaltern)	✗	✓
Benutzerdefinierte Platzhalter im Template	✗	✓
Grid-Spalten-Footer exportieren	✗	✓
Titelzeile ueber alle Spalten (auto-merge)	✗	✓
Mehrzeilige Header / Joined Headers	✗	✓
Spalten-Reihenfolge im Export ueberschreiben	✗	✓
Sheet-Nummer waehlen (bei Multi-Sheet-Templates)	✗	✓
Auto-Size Columns	✗	✓

2.5 UI-Integration & Performance

Feature	xlsbuilder	Flowingcode
Fertige Download-Buttons (auto an Grid-Footer)	✗	✓
Concurrent-Download-Throttling (Semaphore)	✗	✓
Performance (25.000 Zeilen, Median)	237 ms	5.565 ms
Arbeitsspeicher-Effizienz	Streaming	Alles im RAM
Benoetigt Vaadin-Session/UI	Nein	Ja

3 Empfehlung

xlsbuilder	Flowingcode
------------	-------------

• Performance zaehlt (~23x schneller) • Echte typisierte Zellen (Datum, Boolean, Formel) • Klickbare Hyperlinks in Excel • Grosse Datenmengen speicherschonend streamen • Sortierung der Tabelle wird uebernommen	• Export soll den aktiven Grid-Filter spiegeln • Hierarchische Daten (TreeGrid) exportieren • Mehrere Formate anbieten (CSV, DOCX, PDF) • Excel-Vorlage mit Platzhaltern befuellen • Fertige UI-Buttons ohne eigenen Code
---	--

4 Technische Details

Projekt	VaadinExcelExport (github.com/MaKnoNet/VaadinExcelExport)
Stack	Java 21, Spring Boot 3.3.5, Vaadin 24.5.3
xlsbuilder	de.makno.xlsbuilder:xlsbuilder:1.0.0 (Gradle Composite Build)
Flowingcode	org.vaadin.addons.flowingcode:grid-exporter-addon:2.5.0
Apache POI	5.4.0
Benchmark	ExcelExporterBenchmarkTest (@Tag("benchmark"))
Ausfuehrung	./gradlew benchmark [-Prows=N]
Datum	4. Juni 2026